

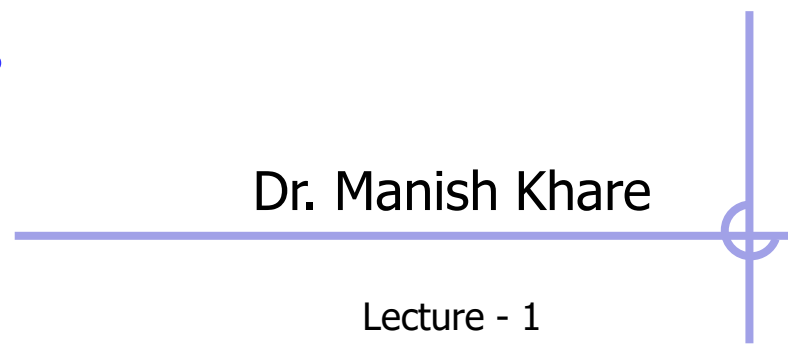
Hash Tables
Trees
Sets
Stack
Linked Lists
Graphs
Sets
Queues
DATA STRUCTURES
Linked Lists
Queues
Hash Tables
Sets
Array
Stack
Queues
Trees
Linked Lists
Graphs
Sets
Array

Data Structures

IT623

Dr. Manish Khare

Lecture - 1



About Me

Dr. Manish Khare

PhD (Computer Science)

Research Interests

Image Processing

Computer Vision

Infrared Imaging

Surveillance System

Manish Khare

Room No.: Faculty Block 4/ 4204

Extn No.: 619

Mail: manish_khare@daiict.ac.in

Introduction



➤ Course Name: Data Structures

➤ Course Id: IT623

➤ Credits: 3(L)-0(T)-2(P)-4(C)

Schedule



➤ Lectures

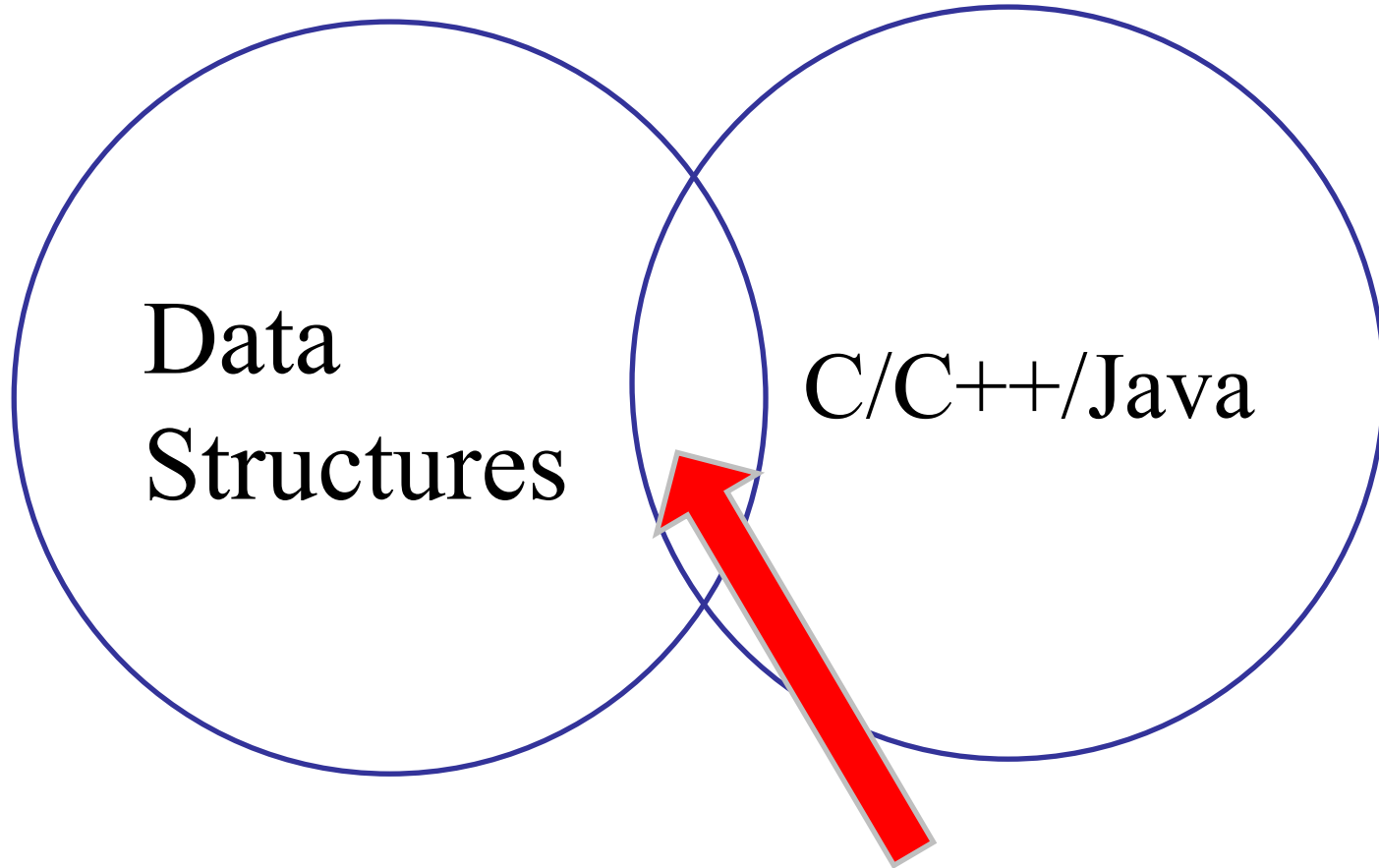
- Monday (10:00 – 10:50)
- Wednesday (10:00 – 10:50)
- Friday (10:00 – 10:50)

➤ CEP 207

➤ Lab

- Schedule and Venue will be announced Later

About the Course



Data Structures

Text and References

- Fundamentals of Data Structures in C, 2nd Edition, E. Horowitz, S. Sahni and Susan Anderson Freed, Universities Press.
- An Introduction to Data Structures with Application, 2nd edition by Jean-Paul Tremblay, Paul Sorenson, Mc Graw Hill publications.
- Data Structures using C: Reema Thareja, Oxford University Press.
- Data Structures: A Pseudocode Approach with C, 2nd Edition, R. F. Gilberg and B.A. Forouzan, Cengage Learning.
- Data Structures using C – A. S. Tanenbaum, Y. Langsam, and M.J. Augenstein, PHI/Pearson Education
- Introduction to Algorithms - Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein

Course Outline

- **Introduction:** Data representation; Linear and non-linear data structures, Algorithm analysis, running time.
- **Strings and Arrays:** String operations - indexing, pattern matching, insertion, deletion, replacement; Array addressing; Binary search; Matrices.
- **Linked Lists:** Linked List traversing, insertion, deletion; Circular Linked List; Two-way Linked List; Hashing.
- **Stack, Recursion, Queue:** Stack push-pop operations; Recursion; Insertion, deletion in Queue; Deque; Priority Queue.
- **Trees:** Binary Tree - Inorder, Preorder, Postorder traversal; Threading; Binary Search Tree, AVL Search Tree, m-Way Search Tree, B-Tree; Heap, Forest.
- **Graphs:** Weighted, directed, connected graph; Adjacency Matrix; Shortest Path Algorithm; Graph operations; Graph algorithms; Depth First Search, Breadth First Search.
- **Sorting:** Quick Sort, Heap Sort, External Sorting- Model for external sorting, Merge Sort.

Lab Assignments

- There will be regular lab assignments, usually once a week, 5 - 8 in all.
- Programming Language: C

Lecture and Lab Material

Classroom Code: om3shez



Join Google Classroom only from Institute Mail ID.

Students who join Classroom from their personal email id, will be removed from Classroom

Course Evaluation Distribution

➤ Evaluation Distribution:

- 1st In-Semester Exam – 15%
- 2nd In-Semester Exam – 15%
- End Semester Exam – 40%
- Class Quizzes/Viva – 15%
- Lab Assignments and Exam – 15%

Some Important Points

- NO mobile phones / laptops Use in Class
- Be Punctual, Be Regular, and Avoid Backlog
- Attendance – class [Minimum 70% Attendance required] for eligible in Exams
- Timely submission of assignments, no late submissions will be allowed
- NO repeat quiz
- Suggestions and feedback welcome

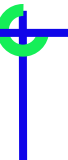
Disclaimer !

- 
- This course will not teach you about
 - Any specific software development technologies like .NET, Java, C, C++
 - The course assumes students exposed to C programming
 - Learn concepts and then problem solving...



Questions???

Data Structures



➤ What is Data Structures?

What is Data Structures?

- A data structure is a specialized format for organizing and storing data. General data structure types include the array, the file, the record, the table, the tree, and so on. Any data structure is designed to organize data to suit a specific purpose so that it can be accessed and worked with in appropriate ways. In computer programming, a data structure may be selected or designed to store data for the purpose of working on it with various algorithms.
- Data Structure is a systematic way to organize data in order to use it efficiently. Following terms are the foundation terms of a data structure.
- **Interface** — Each data structure has an interface. Interface represents the set of operations that a data structure supports. An interface only provides the list of supported operations, type of parameters they can accept and return type of these operations.
- **Implementation** — Implementation provides the internal representation of a data structure. Implementation also provides the definition of the algorithms used in the operations of the data structure

Data Structures



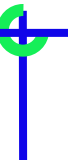
➤ What is Data Structures?

➤ Characteristics of a Data Structures?

Characteristics of a Data Structures?

- **Correctness** – Data structure implementation should implement its interface correctly.
- **Time Complexity** – Running time or the execution time of operations of data structure must be as small as possible.
- **Space Complexity** – Memory usage of a data structure operation should be as little as possible.

Data Structures

- 
- What is Data Structures?
 - Characteristics of a Data Structures?
 - Why do we need to learn data structures?

Why do we need to learn data structures?

- Programs are comprised of two things: data and algorithms. The algorithms describe the way the data is to be transformed. The reason for learning about data structures is because adding structure to our data can make the algorithms much simpler, easier to maintain, and often faster.
- As applications are getting complex and data rich, there are three common problems that applications face now-a-days.
- **Data Search** – Consider an inventory of 1 million (10^6) items of a store. If the application is to search an item, it has to search an item in 1 million (10^6) items every time slowing down the search. As data grows, search will become slower.
- **Processor speed** – Processor speed although being very high, falls limited if the data grows to billion records.
- **Multiple requests** – As thousands of users can search data simultaneously on a web server, even the fast server fails while searching the data.

Data Structures

- 
- What is Data Structures?
 - Characteristics of a Data Structures?
 - Why do we need to learn data structures?
 - What is Data Types?

What is Data Types?

➤ A data type (in computing terms) is a set of data values having predefined characteristics. You will have encountered data types during computer programming classes. Example data types would be integer, floats, string and characters. Generally, in all programming languages we have a limited number of such data types. The range of values that can be stored in each of these data types is defined by the language and the computer hardware that you are using the high level language on.

➤ Scalar Data types

A simple (scalar) data type consists of a collection of ordered values and a set of operations that can be performed on those values. The C, C++ and Java programming languages refer to scalar types as primitive data types.

A scalar variable can hold only one piece of data at a time. So in C, C++ and Java scalar data types include int, char, float and double, along with others. Scalar variables of the same type can be arranged into ascending or descending order based on the value.

➤ Structured data types

Structured data types hold a collection of data values. This collection will generally consist of the primitive data types. Examples of this would include arrays, records (structs), classes and files. These data types, which are created by programmers, are extremely important and are the building block of data structures. So what exactly is a data structure?

Data Structures

- 
- What is Data Structures?
 - Characteristics of a Data Structures?
 - Why do we need to learn data structures?
 - What is Data Types?
 - Basic Terminology in Data Structures?

Basic Terminology in Data Structures?

- **Data** – Data are values or set of values.
- **Data Item** – Data item refers to single unit of values.
- **Group Items** – Data items that are divided into sub items are called as Group Items.
- **Elementary Items** – Data items that cannot be divided are called as Elementary Items.
- **Attribute and Entity** – An entity is that which contains certain attributes or properties, which may be assigned values.
- **Entity Set** – Entities of similar attributes form an entity set.
- **Field** – Field is a single elementary unit of information representing an attribute of an entity.
- **Record** – Record is a collection of field values of a given entity.
- **File** – File is a collection of records of the entities in a given entity set.



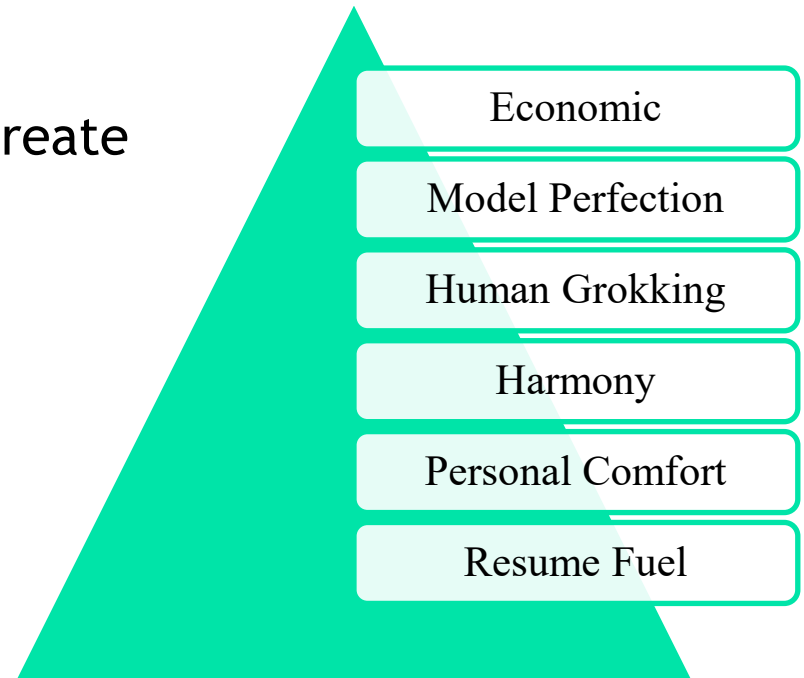
*What is the Purpose of | Why we do
Programming?*

Arguments...

The purpose of programming is to automate processes

The practice of programming is one of teaching the computer to do something

The purpose of programming is to create



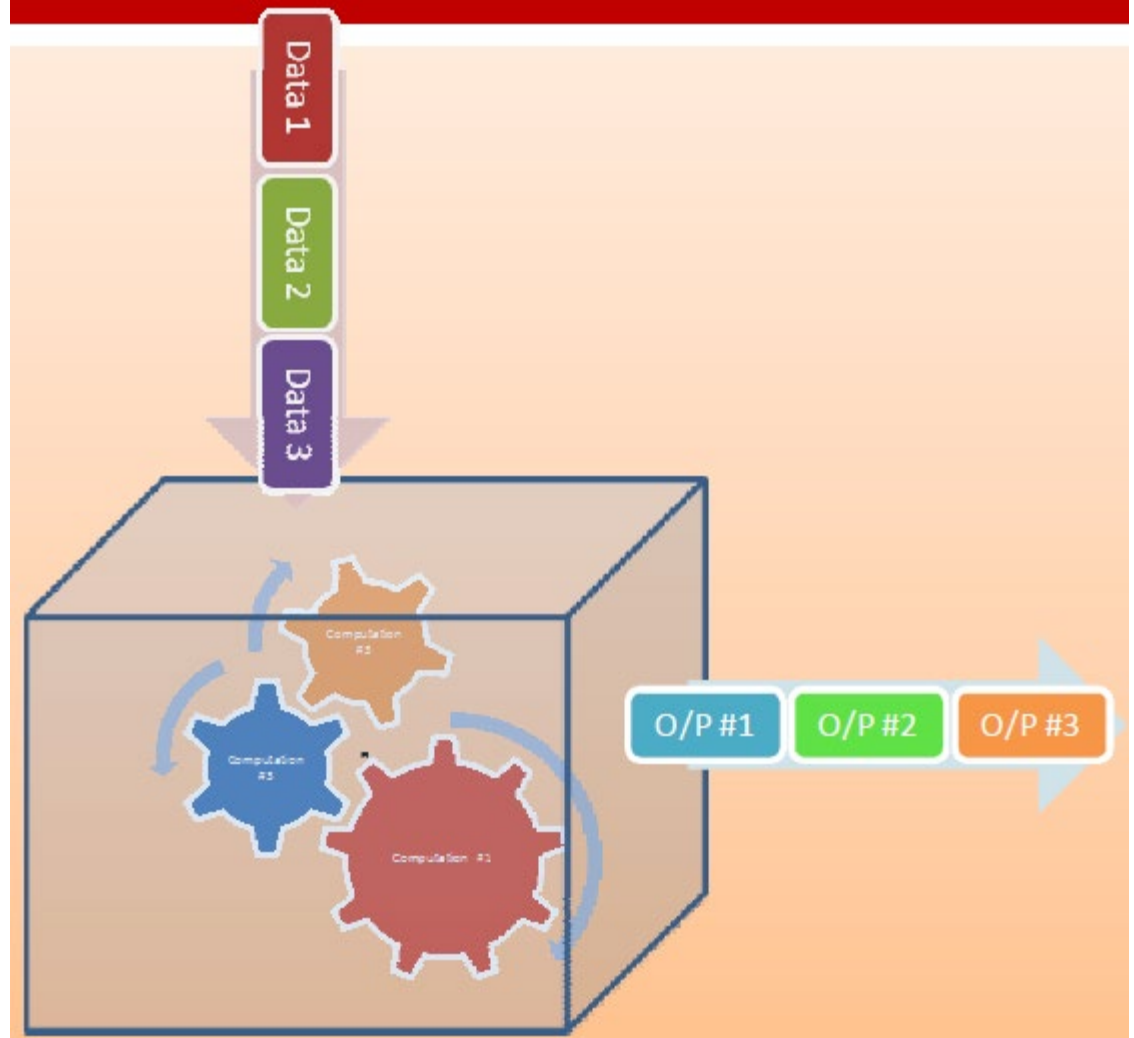
Computer Engineering is about...

Solving problems by **writing programming solutions**

- Not just any solution but the best possible solution of the problem
 - a. Characteristics of the best solution
 - b. Efficient (performance or speed)
 - c. Cost effective
 - d. Scalable
 - e. Useful
 - f. Correct
 - g. Robust
 - h. Secure
 - i. Modifiable (change)

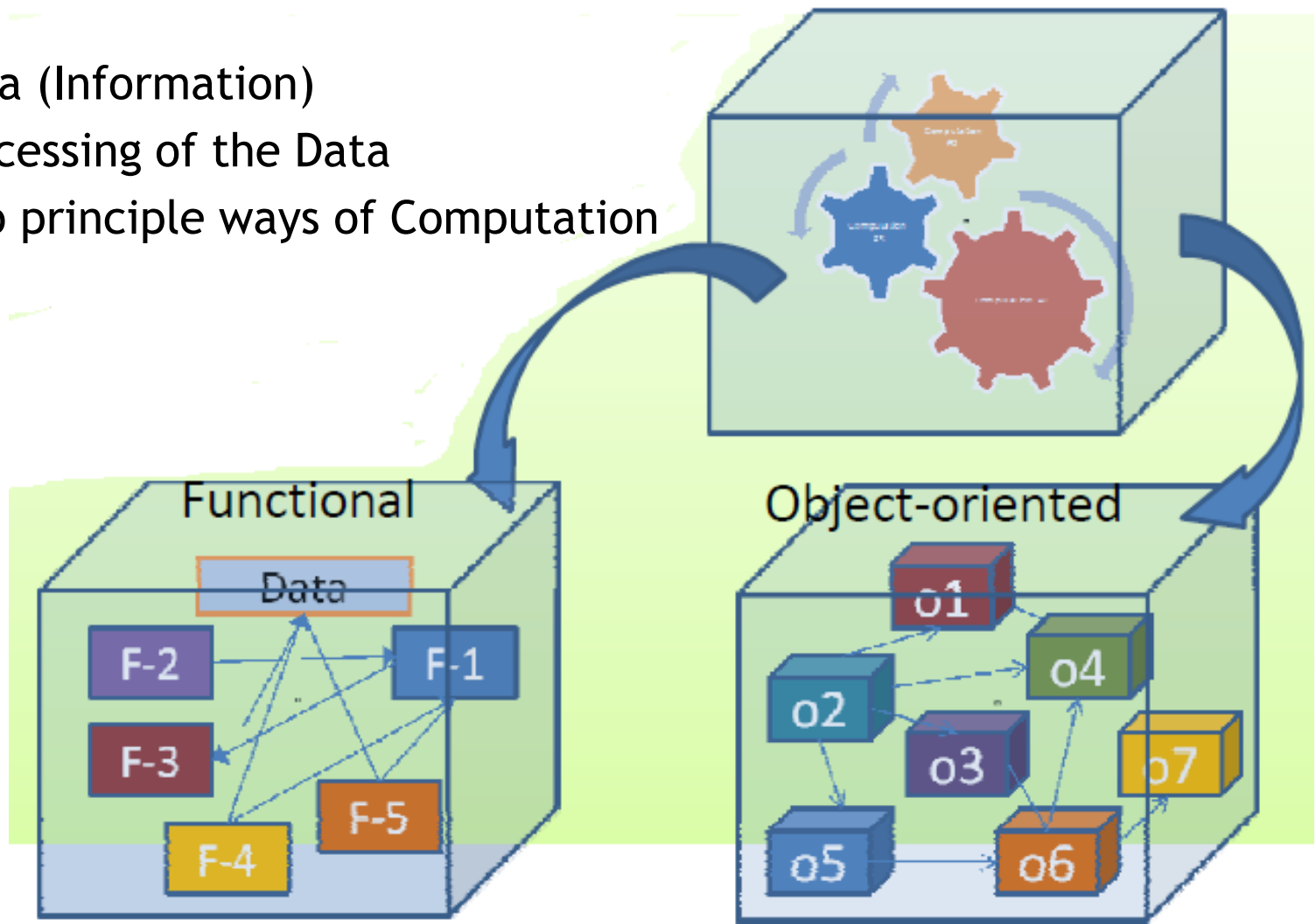
A Programming Solution

- Inputs
 - Keyboard (user)
 - Mouse (user)
 - Network (system)
 - Devices (system)
 - Other programs
- Computations
- Outputs
 - Monitor
 - Network
 - Devices
 - Other programs
- GUI



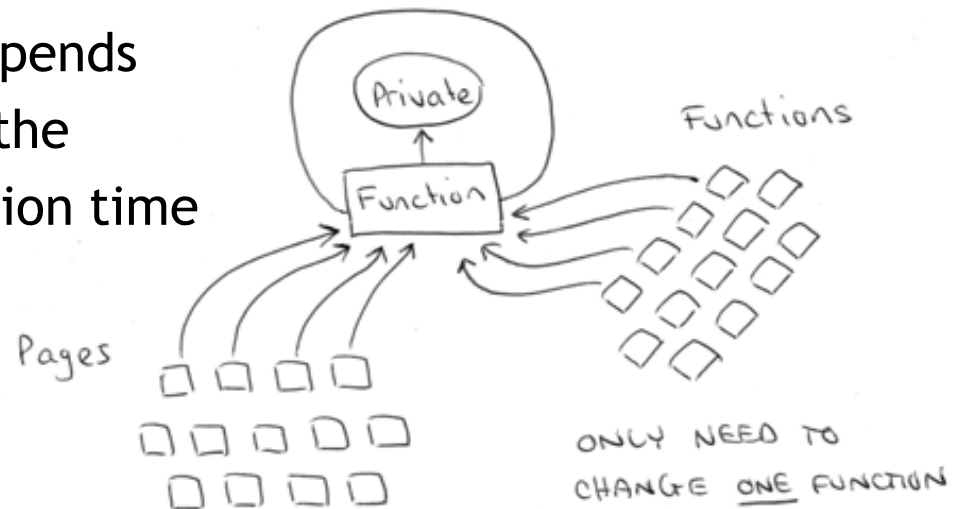
Computation

- Data (Information)
- Processing of the Data
- Two principle ways of Computation



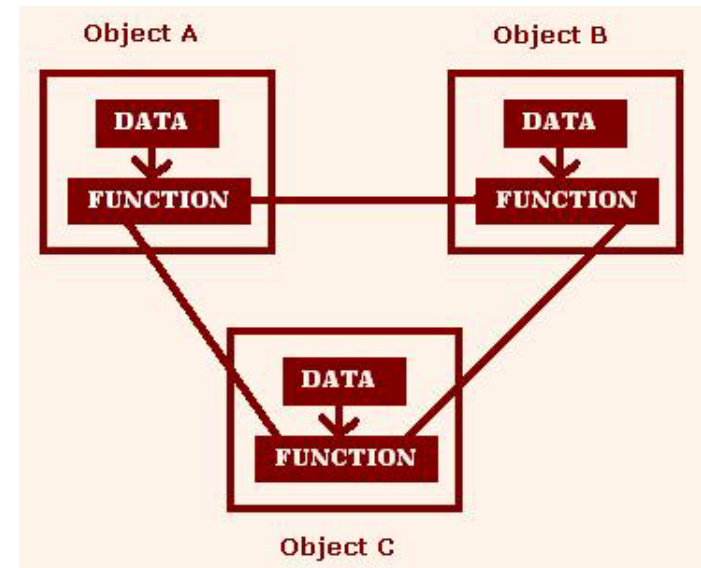
Functional

- Basic elements of functional programming are:
- Variables in the mathematical sense (named values)
- Functions in the mathematical sense (mapping values to values)
- The result of a function depends only on its arguments and the Context at function definition time



Object-Oriented

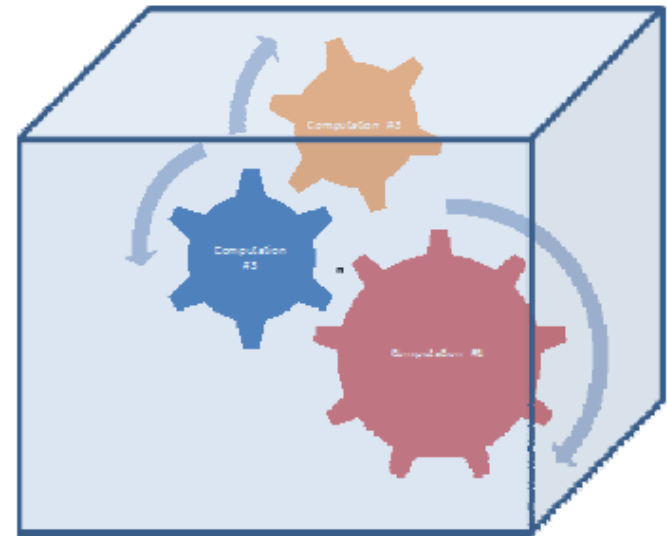
- Basic elements of Object-Oriented Programming (OOP) are:
- Objects (data) with - typically - non-constant state
- Variables are storage locations in memory that may hold different values at different times.
- Methods (operations) on objects (data) that may or may not change the objects state.
- OOP is basically imperative programming with an additional *level of abstraction* regarding data and functions.



Improving Computation

Data Structures

- Choosing suitable data organization
- Having efficient data storing and retrieving mechanism



Solution Characteristics

✓ Efficient (performance or speed)
✓ Cost effective
✓ Scalable

× Useful
× Correct
× Robust
× Secure
× Modifiable

Problem
+
Language
+
Data Structure

